

Sprout — Product Development Plan

Oct 3, 2026 · @Ashmith

Overview

Sprout is a personal learning agent on Agentverse that remembers what you know, so every new chat starts where your last one ended. It targets the Fetch.ai track at MHacks 2026 and is used entirely inside ASI:One.

One-line pitch: A study buddy that grows with you: it learns your curriculum, tracks what you've mastered, and teaches the next thing in the format that works for you.

Problem. Students spend the first part of every AI study session re-explaining context. The model doesn't know their course, misreads course-specific terms, over- or under-explains, and skips prerequisites it can't see. That setup is lost the moment a new chat opens, so the cost repeats every session.

Insight. Learning is cumulative. A tutor that knows exactly which concepts you've mastered, which are shaky, and what depends on what can pick the right next step without being told.

Theme fit. MHacks 2026's theme is Digital Garden, "build something that grows." Sprout's knowledge map is literally a garden: concepts sprout, grow as mastery rises, and wilt when they're due for review.

"Sprout" is a working name.

Target users

The launch user is a college student in a structured course with a syllabus, exams, and cumulative material. Demo the CS student persona; the others show reach.

Persona	Situation	What Sprout does for them
CS sophomore	Data structures course, midterm in a week, uses AI chat daily	Knows the syllabus and which topics are shaky; builds targeted practice before the exam

Persona	Situation	What Sprout does for them
Pre-med student	Huge memorization load across bio and chem	Spaced-repetition flashcards scheduled before forgetting kicks in
Self-learner	Teaching themselves ML from online courses, no instructor	Builds the prerequisite map so they don't skip linear algebra they need
High school AP student	Following a fixed College Board curriculum	Curriculum-aware explanations at the right level

Not in scope for the hackathon: instructors, group study, and K-8 learners.

Core features

The MVP is five features that together prove one claim: open a new chat and Sprout already knows where you are. Everything else is stretch.

Feature	What it does	Priority
Curriculum ingestion	User pastes a syllabus or notes; Sprout extracts concepts and builds a prerequisite map	MVP
Diagnostic quiz	Short adaptive quiz on day one to set starting mastery per concept	MVP
Persistent knowledge state	Mastery estimate per concept, saved per user and updated after every answer	MVP
Next-step teaching	Picks the weakest concept whose prerequisites are solid, then explains it with course context	MVP
Interactive study tools	Quizzes, flashcards and worked examples rendered as ASI Interactive Cards	MVP
Format adaptation	Learns which formats (diagram, worked example, flashcards, analogy) improve this user's scores	Stretch, high value

Feature	What it does	Priority
Spaced review scheduling	Flags concepts due for review based on predicted forgetting	Stretch
Knowledge garden view	Web page that draws the concept map as a growing garden	Stretch, theme bonus
Calendar / Canvas sync	Pulls exam dates to prioritize what to study	Stretch
Paid study packs	Exam-prep mode unlocked via the Payment Protocol	Stretch, Fetch bonus

Cut order if time runs short: Calendar sync, then garden view, then spaced review. Never cut format adaptation's simplest version; it's the strongest ML story.

User experience

Users talk to Sprout in ASI:One, Fetch.ai's chat app. No custom frontend is needed for the core flow; rich interactions use ASI Interactive Cards inside the chat.

Discovery. A student asks ASI:One something like "help me study for my data structures midterm." ASI:One routes the request to Sprout based on its Agentverse listing. Users can also open Sprout from its Agentverse profile.

First session

1. Sprout greets the user and shows a form card: course name, current unit, exam date.
2. User pastes the syllabus or notes. Sprout replies with a summary card of the concepts it found and asks the user to confirm.
3. A 5-8 question diagnostic runs as tappable quiz cards.
4. Sprout shows a knowledge snapshot card: each concept marked solid, shaky, or not started.
5. Sprout teaches the first gap and ends with a two-question check.

Returning session (the key moment)

1. User opens a brand-new ASI:One chat and says "let's keep going."
2. Sprout recognizes the user's address and loads their saved state.
3. It opens with where they left off, what's due for review, and days until the exam.
4. It teaches the next concept in the format that has worked best for this user.

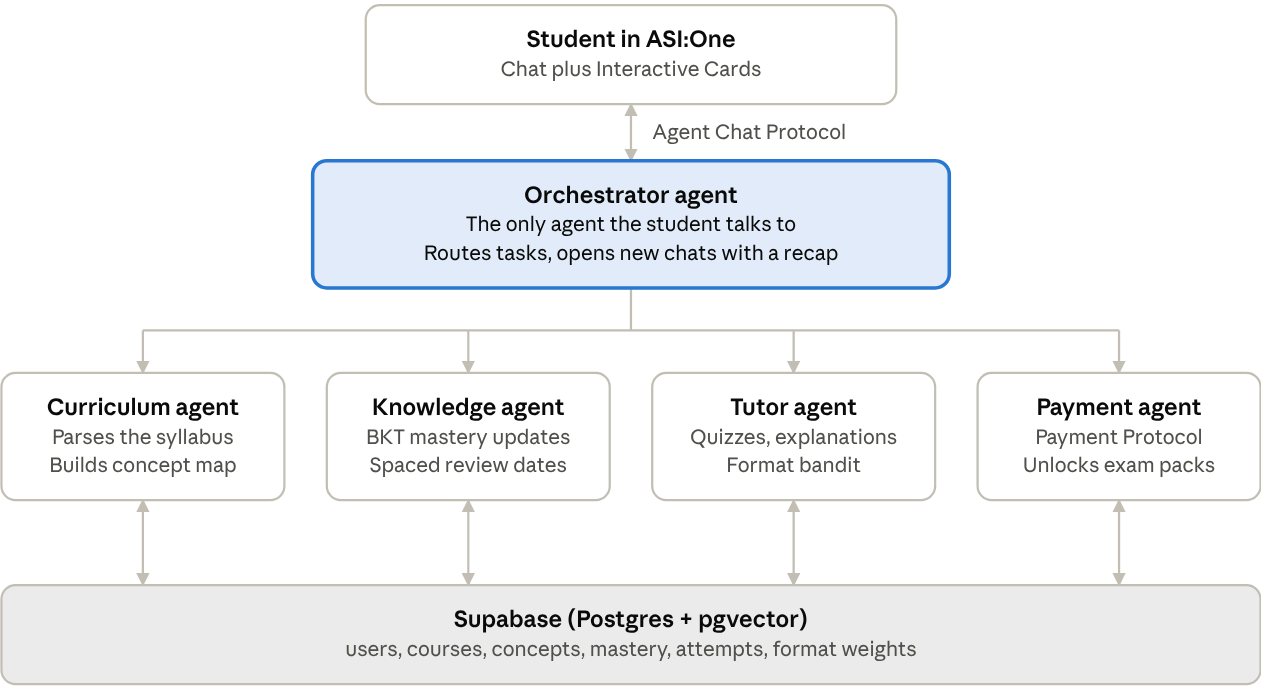
Card types

Card	Used for
Form	Onboarding, adding a course, updating exam dates
Quiz question	Diagnostics and checks, tappable answers
Knowledge snapshot	Mastery per concept at a glance
Flashcard carousel	Memorization-heavy concepts
Worked example	Step-by-step problems
Review reminder	Concepts due today

System architecture

Sprout is five agents on Agentverse: one orchestrator the student talks to, and four specialists it delegates to. All of them read and write one shared database, which is what lets a brand-new chat pick up where the last one ended.

One orchestrator talks to the student; four specialists share one memory



Sprout architecture · 1 orchestrator, 4 specialist agents, 1 database

The orchestrator calls the ASI:One LLM for conversation; the curriculum and tutor agents call it to extract concepts and generate content. Any framework works, but uAgents is the shortest path to Agentverse registration and the Chat Protocol.

ML models and adaptive engine

Four lightweight models turn Sprout from a chatbot with memory into an adaptive tutor. Each needs no training data and fits in a 24-hour build; the LLM handles language, the models handle decisions.

Model	Job	Inputs → output	Build cost
Bayesian Knowledge Tracing (BKT)	Estimate mastery per concept	Each answer (right/wrong) → P(mastered)	~2 hours, pure Python
Embedding concept graph	Find concepts and prerequisite links in the curriculum	Syllabus text → concept nodes + edges	~3 hours, embeddings + LLM extraction
Contextual bandit (Thompson sampling)	Learn which teaching format works for this user	Format used + quiz result after → format weights	~2 hours
Spaced repetition (FSRS or SM-2)	Predict forgetting and schedule reviews	Review history → next review date	~1 hour with an open-source library
ASI:One LLM	Reason, explain, generate quizzes and cards	Prompt + user state → content	Provided

BKT update. After each answer, mastery is updated with Bayes' rule using four parameters per concept: prior, learn rate, slip, and guess. Start with standard defaults (prior 0.2, learn 0.15, slip 0.1, guess 0.25) and tune if time allows.

$$P(L_t \mid \text{correct}) = \frac{P(L_t)(1-s)}{P(L_t)(1-s) + (1-P(L_t))g}$$

Then $P(L_{\text{next}}) = P(L \text{ given evidence}) + (1 - P(L \text{ given evidence})) \times \text{learn rate}$.

Choosing what to teach next. Pick the concept with the lowest mastery whose prerequisites are all above 0.7. This is what makes Sprout respect that knowledge builds on knowledge.

Framing learning styles honestly. Research on fixed learning styles ("I'm a visual learner") is weak, and an education-savvy judge may say so. Pitch format adaptation as an experiment the agent runs on each student: it tries formats and keeps whatever measurably improves their scores. Users can still state a preference; it sets the bandit's starting weights.

Demo-visible ML. Show the knowledge snapshot updating live after quiz answers, and a line like "worked examples have raised your scores 30% more than flashcards."

Data model and memory

Each user is identified by the sender address AS!One passes to the agent, and everything Sprout knows hangs off that key. Use Supabase (Postgres + pgvector) so memory survives agent restarts and the garden view can read it; fall back to the agent's built-in storage if setup stalls.

Table	Key fields	Purpose
users	address, display name, created at	One row per learner
courses	user, name, exam date, raw syllabus	A user can have several courses
concepts	course, name, summary, embedding	Nodes of the knowledge map
prerequisites	concept, requires concept, confidence	Edges of the knowledge map
mastery	user, concept, P(mastered), last seen, next review	BKT state plus review schedule
attempts	user, concept, format used, correct, timestamp	Evidence log feeding BKT and the bandit
format_weights	user, format, alpha, beta	Bandit state per user
sessions	user, started, summary	Short recap used to open the next chat

Privacy. Store only course material and learning data. No grades, emails, or school IDs. Let users say "forget this course" to delete its rows.

Fetch.ai track alignment

Sprout meets every mandatory requirement in the MVP and targets four of the six bonus areas. Prizes are \$1,250, \$750 and \$500, each with an internship interview ([hackpack](#)).

Requirement	How Sprout meets it
At least one agent on Agentverse	Five agents registered, Innovation Lab badge in each README
Agent Chat Protocol	Orchestrator implements ACP; it's the only agent users talk to
Discoverable and usable via ASI:One	Clear Agentverse name, description and keywords for study requests
Meaningful tool execution or orchestration	Curriculum parsing, quiz generation, BKT updates, database writes, multi-agent routing
Primary workflow inside ASI:One	Onboarding, quizzes and teaching all run as chat plus Interactive Cards
Public GitHub repo with run instructions	README lists agent names and addresses

Bonus areas targeted: multi-agent collaboration, Interactive Cards, Payment Protocol, and an agent that keeps working after the hackathon. Error handling is a cheap fifth: retry failed tool calls and tell the user plainly when something failed.

Against the judging weights

Criterion	Weight	Sprout's angle
Functionality & technical implementation	25%	Live BKT updates, working cross-session memory
Use of Fetch.ai technology	20%	Agentverse, ACP, Payment Protocol, Interactive Cards
Innovation & creativity	20%	Knowledge tracing + format bandit, not just an LLM with notes

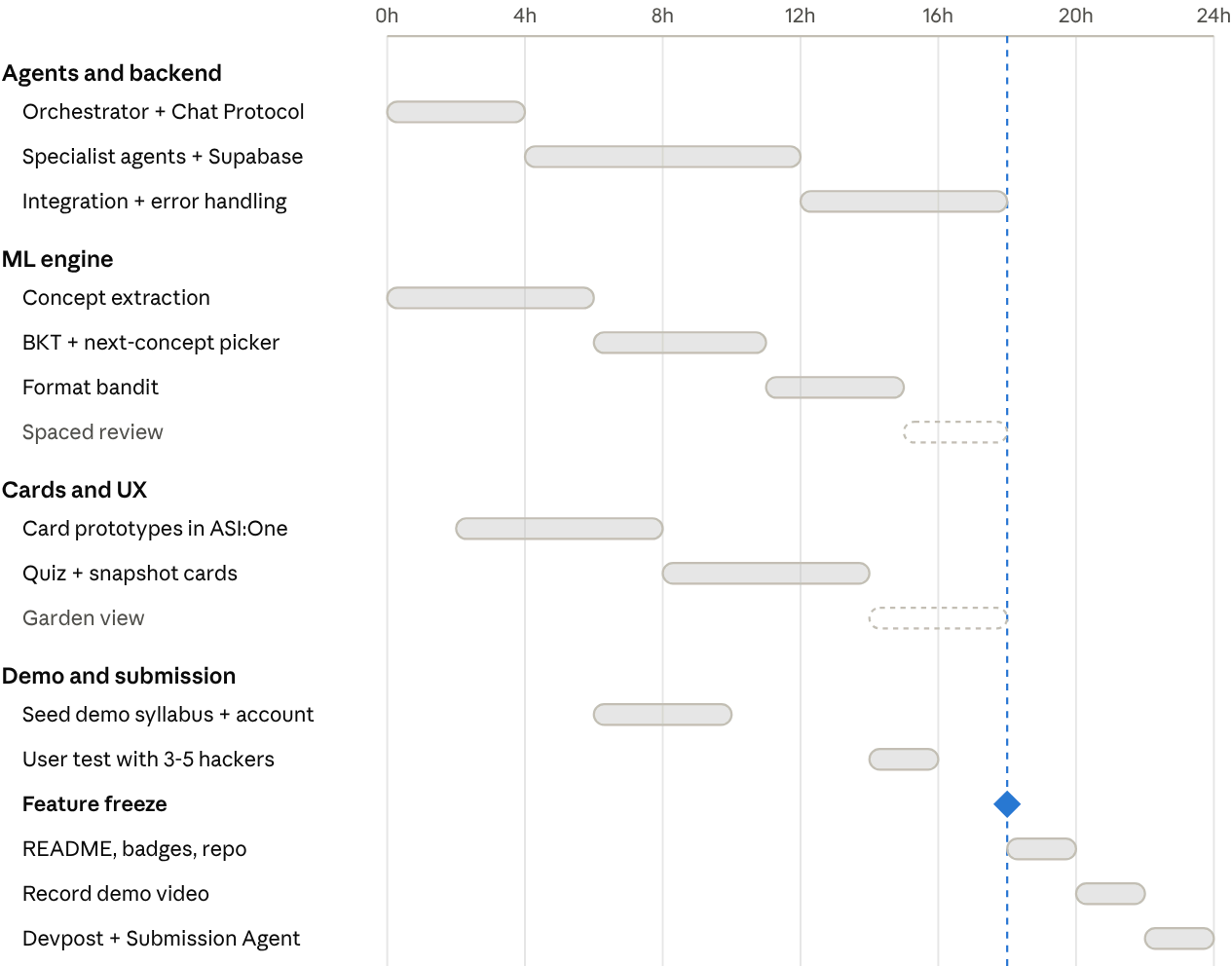
Criterion	Weight	Sprout's angle
Real-world impact	20%	Every student who uses AI to study feels this pain
UX & presentation	15%	The new-chat-already-knows-me moment

Build plan and team roles

The plan splits the 24 hours into four workstreams so the core loop (chat, memory, BKT) works by hour 12 and nothing new starts after hour 18. It assumes a team of four, one person per workstream; with three, merge Cards and UX into Demo.

Core loop works by hour 12; features freeze at hour 18

Hours from hacking start. Dashed bars are stretch work.



24-hour build plan · 4 workstreams, feature freeze at hour 18

Workstream	Owns	Done when
Agents and backend	uAgents setup, Chat Protocol, Agentverse registration, Supabase, agent routing	A new chat loads saved state for a returning user
ML engine	Concept extraction, BKT, next-concept picker, format bandit	Mastery updates after every answer
Cards and UX	Interactive Cards, prompts, tone, optional garden view	Full onboarding and quiz flow runs in ASI:One
Demo and submission	Demo data, user test, README, video, Devpost	Every checklist item ticked

Checkpoints: hour 4, "hello" works through ASI:One; hour 12, a returning user gets a recap; hour 18, freeze and fix bugs only.

Demo script

The demo runs about 4 minutes, fitting the 3–5 minute video requirement, and is built around one moment: a brand-new chat that already knows the student.

1. **Hook (0:00–0:30).** Show a real ChatGPT-style chat where a student spends a long message re-explaining their course. "You do this every single time."
2. **Onboarding (0:30–1:15).** In ASI:One, paste a real data structures syllabus. Sprout returns the concept map and runs three diagnostic quiz cards.
3. **Live ML (1:15–2:00).** Answer one question wrong on purpose. The knowledge snapshot card updates; Big-O drops to shaky. Sprout teaches it with a worked example.
4. **The moment (2:00–3:00).** Close the chat. Open a new one. Type "let's keep going." Sprout opens with what's shaky, what's due, and days to the exam.
5. **Adaptation (3:00–3:30).** Show the format insight: worked examples work better than flashcards for this student, so it switched.
6. **Garden + close (3:30–4:00).** Flash the knowledge garden view if built. Close on the line "A study buddy that grows with you."

Pre-seed a demo account with two days of history so step 4 is guaranteed to work live. Record a backup video before judging.

Monetization

Sprout is free for one course and charges for exam-prep power features through Fetch.ai's Payment Protocol. A credible model earns bonus points and is scored under Use of Fetch.ai Technology.

Tier	Price idea	Includes
Free	\$0	One course, knowledge tracking, daily study session
Exam pack	Small one-time payment per exam	Full mock exam, weak-spot drill plan, review schedule up to exam day
Semester	Low monthly price	Unlimited courses, format adaptation insights, garden view

For the demo, show one paid unlock end to end: the user asks for a mock exam, Sprout sends a payment request, and after payment it delivers the exam. Prices are placeholders to set as a team.

Risks and mitigations

The biggest risk is being judged as "just a chatbot"; every mitigation below pushes visible action and visible ML into the demo.

Risk	Likelihood	Mitigation
Looks like a chatbot wrapper	High	Lead with live mastery updates and the new-chat moment, not with explanations
ASl:One doesn't route to Sprout in the demo	Medium	Test discovery early; demo from the Agentverse profile link as backup
Interactive Cards take longer than expected	Medium	Ship plain-text quizzes first, upgrade to cards after the core loop works
Concept extraction is messy on real syllabi	Medium	Let users confirm or edit the concept list; pre-test on the demo syllabus

Risk	Likelihood	Mitigation
Agents lose state on restart	Medium	Store everything in Supabase, not in memory
Judges challenge learning styles	Medium	Frame it as measured format adaptation (see ML section)
Live demo fails	Low-medium	Seeded demo account plus a recorded backup video

Success metrics

Hackathon success is a working cross-session demo; product success is less setup time and better quiz scores.

Metric	Hackathon target	Long-term target
Setup messages before useful help, returning user	0	0
Time from "let's keep going" to first useful content	Under 10 seconds	Under 5 seconds
Mastery estimate updates after each answer	Every answer, visible in demo	Every answer
Quiz score lift on a concept after teaching	Shown once in demo	Measured across users
Weekly returning users	n/a	40% of sign-ups

Run a quick test with 3–5 hackers on Saturday night: time how long their first session takes and whether the second session needs any re-explaining.

Submission checklist

All items come from the Fetch.ai backpack; missing any of the first six makes Sprout ineligible.

- ☐ All agents registered on Agentverse

- ☐ Orchestrator implements the Agent Chat Protocol
- ☐ Sprout is discoverable and usable through ASI:One
- ☐ Full primary workflow works inside an ASI:One conversation
- ☐ Public GitHub repo with run and test instructions
- ☐ README lists every agent's name and address, plus any extra resources needed
- ☐ Innovation Lab and hackathon badges in each agent's README
- ☐ Demo video, 3–5 minutes
- ☐ Project submitted on Devpost
- ☐ Submission registered through the MHacks ASI:One Submission Agent
- ☐ Free ASI:One Pro and Agentverse Premium claimed with code MHACKS26MHACKSAV

Post-hackathon roadmap

If Sprout places or gets traction, the next step is a pilot with real students in one course this semester.

1. **First month:** Keep the agents running on Agentverse, fix demo shortcuts, and pilot with 20–30 students in one large intro course.
2. **Next 2–3 months:** Canvas and Google Calendar sync for exam dates and assignments, PDF lecture-slide ingestion, and a fuller garden web app.
3. **Later:** Tune BKT parameters from real attempt data, share concept maps across students in the same course, and add an instructor view of class-wide weak spots.

Sources

- [Fetch.ai MHacks 2026 hackpack](#)
- [Fetch.ai MHacks 2026 event page](#)